

Theoros security specification

Source: docs/SECURITY.md, commit 47c1211 + uncommitted changes, generated 2026-10-08

Encryption and authentication of Theoros telemetry and logs: threat model, cryptographic design, exact wire and file formats, key management, operating procedures and test vectors. Detailed enough to write a compatible receiver or decoder from this document alone.

Item	Value
Applies to	Firmware 0.2.0 and later (<code>firmware/tracker</code> , <code>firmware/receiver</code>)
Packet format	Version 2 (version 1 was unencrypted and is no longer accepted)
Log format	Encrypted log v1
Status	Implemented and host-tested; not yet run on hardware
Reference code	<code>firmware/lib/theoros-core/src/theoros/</code> (<code>crypto</code> , <code>keys</code> , <code>packet</code> , <code>seclog</code> , <code>security</code>), <code>firmware/tools/theoros.py</code>

Contents

1. [Summary](#)
2. [Threat model](#)
3. [Design overview](#)
4. [Cryptographic primitives](#)
5. [Key hierarchy](#)
6. [Nonces and the message counter](#)
7. [LoRa packet format, version 2](#)
8. [Receiver processing](#)
9. [Encrypted log format](#)
10. [Key storage on the device](#)
11. [Provisioning](#)
12. [Operating procedures](#)
13. [Test vectors](#)
14. [Verification](#)
15. [Limitations](#)
16. [Hardening roadmap](#)

1. Summary

- Every LoRa packet a tracker sends and every record it writes to its SD card is encrypted and authenticated with **AES-128-CCM** (RFC 3610), the construction used by LoRaWAN, Zigbee and Bluetooth LE.
- Each tracker has its own key, derived from a **master key** that only the receiver and the owner's PC hold. A captured tracker exposes only its own data.
- The receiver rejects forged, altered, replayed and stale packets.
- A tracker without a key **never transmits**. It logs in plaintext and warns, so the board can still be brought up.
- Not protected: the fact that a tracker transmits, keys against someone who physically dumps a board's flash (until eFuse hardening, section 16), and data on the receiver, which is the trusted end.

2. Threat model

Assets

Asset	Where
Tracker positions and times	LoRa packets, tracker SD card, receiver outputs
Integrity of received positions	Receiver log, display, USB output
Master key	Owner's PC (file), receiver flash
Device keys	Each tracker's flash

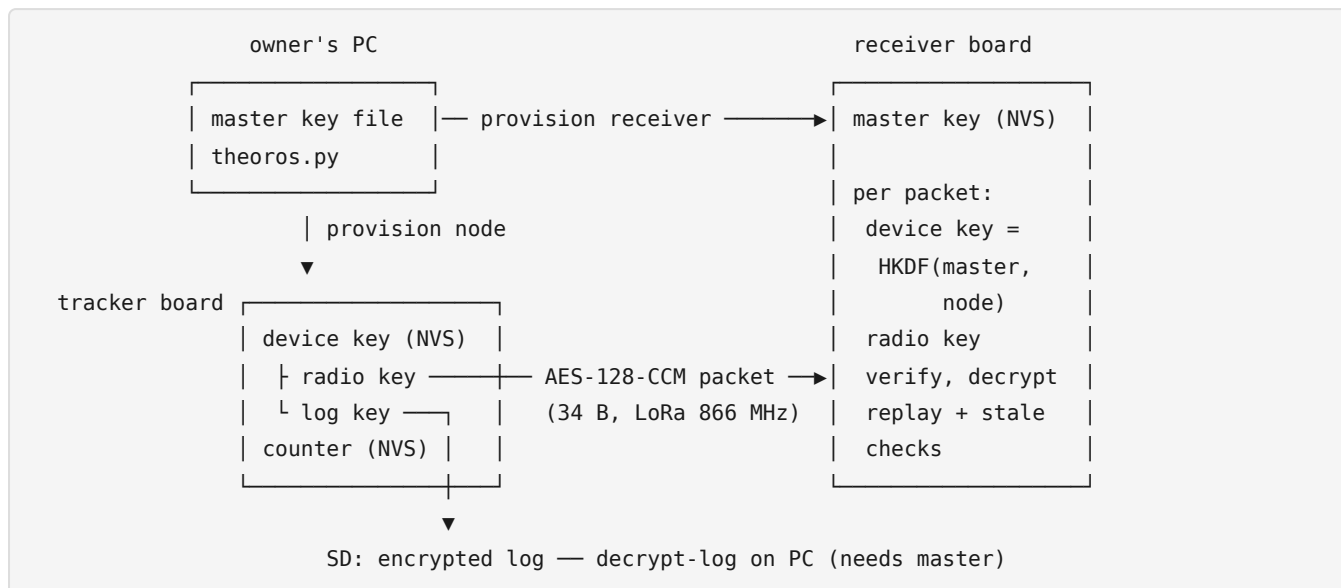
Adversaries

Adversary	Capability	Outcome
Passive listener	Receives all LoRa traffic (any SX127x board or SDR)	Learns node IDs, counters, timing and signal strength; not positions
Active radio attacker	Transmits arbitrary packets, records and replays real ones	Cannot inject or alter data; replays rejected (see 8 and 15 for the post-reboot window)
Card thief	Takes or copies a tracker's SD card	Gets ciphertext only
Tracker capture	Has a tracker, can dump its flash	Gets that tracker's device key: its past and future data. Not the master, not other trackers
Receiver capture	Has the receiver, can dump its flash	Gets the master key: all trackers. Protect the receiver accordingly
LAN attacker	On the same WiFi while service mode is on	Can see the web password and status page; downloaded tracker logs remain encrypted

Out of scope

- Hiding that a transmitter exists, or its location (direction finding).
- Jamming (denial of service).
- Side-channel attacks on the ESP32-S3 (power analysis, fault injection).
- A compromised owner PC.

3. Design overview



4. Cryptographic primitives

Use	Algorithm	Parameters
Encryption and authentication	AES-128-CCM, RFC 3610 / NIST SP 800-38C	128-bit key, 13-byte nonce ($L = 2$), 8-byte tag ($M = 8$)
Key derivation	HKDF-SHA256, RFC 5869	Salt <code>theoros</code> , outputs 16 bytes
Key fingerprint	HKDF-SHA256	4 bytes, shown as 8 hex digits

Why CCM: designed for short, constrained radio messages; standard in LoRaWAN, 802.15.4 and BLE; supported by the ESP32-S3's AES hardware through mbedTLS. AES-GCM or ChaCha20-Poly1305 would be equally secure with the same nonce discipline.

Why an 8-byte tag: a forgery attempt succeeds with probability 2^{-64} . Over LoRa, where each attempt costs ~ 250 ms of airtime, that is beyond reach. LoRaWAN uses 4 bytes.

Implementations: device: mbedTLS from ESP-IDF 5.5 (`crypto_mbedtls.cpp`, hardware AES). Host tests: OpenSSL (`firmware/tests/crypto_openssl.cpp`). PC tool: Python `cryptography`. HKDF, nonce construction and formats are shared portable C++ (`crypto.cpp`, `keys.cpp`, `packet.cpp`, `seclog.cpp`).

5. Key hierarchy

```
master (16 B, random, generated on the owner's PC)
├─ device key = HKDF(salt="theoros", IKM=master, info="device v1" || node)  per tracker
├─ radio key = HKDF(salt="theoros", IKM=device, info="radio v1")           LoRa packets
├─ log key   = HKDF(salt="theoros", IKM=device, info="log v1")             SD log lines
fingerprint(k) = HKDF(salt="theoros", IKM=k, info="fingerprint v1", L=4)
```

Item	Exact bytes
Salt	ASCII <code>theoros</code> (7 bytes)
node	16-bit node ID, little endian (2 bytes)
Info strings	ASCII, no terminator: <code>device v1</code> , <code>radio v1</code> , <code>log v1</code> , <code>fingerprint v1</code>

Item	Exact bytes
Output length	16 bytes for keys, 4 bytes for fingerprints
Fingerprint display	The 4 bytes as 8 lowercase hex digits, big endian

Node ID: the last two bytes of the ESP32-S3 factory MAC (`ESP.getEfuseMac() >> 32`), shown by the console `status` command.

Separation: radio and log keys differ, so the two uses never share keystream even with equal counters. Nonces also carry a domain byte (section 6), so they stay distinct even if keys were ever shared.

Fingerprints identify a key without revealing it, so a board's key can be checked against the provisioning output. They are public.

6. Nonces and the message counter

Nonce (13 bytes)

Bytes	Content
0-3	Message counter, u32 little endian
4-5	Node ID, u16 little endian
6	Domain: <code>0x52</code> (R) radio packet, <code>0x4C</code> (L) log record
7-12	Zero

CCM's confidentiality fails completely if a nonce repeats under one key, so the counter must **never repeat** for a tracker.

Counter rules (`security.cpp`)

- One counter per tracker, shared by packets and log records, incremented for every encryption.
- **Persisted in blocks.** NVS (namespace `theoros-sec`, key `ctr`) holds the first value not yet handed out. When the running value reaches it, it is advanced by 4096 before any value from the new block is used. After a reboot, counting resumes from the stored value; the rest of the previous block is skipped. About one flash write per hour.
- **Write failure is fatal for that message.** If the NVS write fails, no value is handed out and nothing is encrypted.
- **Never reset by key changes.** `key set` and `key clear` leave it alone.
- **Flash erase.** With no stored counter (new board, or full flash erase), the tracker waits until its clock is set from GNSS or NTP and starts at $(\text{unix_time} - 1767225600) \times 4$, i.e. 4 per second since 2026-01-01 UTC. The tracker consumes at most ~ 1.1 values per second (1 log record + 0.1 packet), so this start is above any value used before the erase, and re-entering the same key after an erase does not repeat nonces. The budget is enforced at compile time (`config.h`).
- **Exhaustion.** At $2^{32} - 1$ the tracker refuses to encrypt. With the time floor that is reached in 2060; re-key before then.
- The tracker checks that the radio can transmit before taking a value, so waiting for the duty-cycle limit does not consume counters.

7. LoRa packet format, version 2

Radio: raw LoRa (not LoRaWAN), 866.0 MHz, 125 kHz bandwidth, SF9, CR 4/5, explicit header, LoRa CRC on, sync word `0x12`. One packet every 10 s.

34 bytes, all integers little endian:

Offset	Size	Type	Field	Protection
0	1	u8	Version, 2	Authenticated (AAD)
1	2	u16	Node ID	Authenticated (AAD)
3	4	u32	Message counter	Authenticated (AAD)
7	4	u32	Unix time from GNSS, 0 if unknown	Encrypted
11	4	i32	Latitude, 1e-7 degrees	Encrypted
15	4	i32	Longitude, 1e-7 degrees	Encrypted
19	2	i16	Altitude, m	Encrypted
21	2	u16	Speed, 0.1 km/h	Encrypted
23	1	u8	Satellites in use	Encrypted
24	1	u8	HDOP × 10, saturating at 255	Encrypted
25	1	u8	Flags	Encrypted
26	8		CCM tag	

Flags: bit 0 fix valid, bit 1 SD logging active, bit 2 PPS seen in the last 1.5 s. Latitude, longitude, altitude and speed are 0 when bit 0 is clear. Packets are sent without a fix too, as a heartbeat.

Sealing: nonce = counter || node || R (section 6); AAD = bytes 0-6; plaintext = bytes 7-25 (19 bytes); output = header || ciphertext || tag.

Airtime: about 247 ms; 2.5% of the time at the 10 s interval. The firmware caps airtime at 10% (1% at +20 dBm).

8. Receiver processing

For each frame with a valid LoRa CRC, in this order (`security::openPacket`):

1. **Format.** Length must be 34 and byte 0 must be 2, else *malformed*.
2. **Key.** Derive device key and radio key for the header's node ID from the master.
3. **Authenticate and decrypt.** CCM verification with AAD = bytes 0-6. Any change to any byte, a wrong key, or a different node ID gives *authentication failed*. No output is produced, and the peer table is not touched, so forged packets cannot evict real trackers from it.
4. **Replay.** If the counter is less than or equal to the highest one accepted from that node, *replay*.
5. **Freshness.** If both the packet's time and the receiver's clock are known and differ by more than 10 minutes, *stale*.
6. **Accept.** Record the counter, log, display, print.

Peer table: up to 16 trackers in RAM, least recently heard evicted. The highest accepted counter per node is saved in NVS (key `p<node hex>`) on the first packet after boot and then at most every 10 minutes, to bound flash wear. Consequently, after a receiver reboot, a recorded packet less than 10 minutes newer than the last save could be accepted once (section 15).

Rejected frames are reported on USB as `rx: dropped <n>-byte frame, <reason>, <rssi> dBm` and counted.

9. Encrypted log format

A tracker with a key writes `LOGnnnn.CSV` as text:

```
# theoros encrypted log v1 node=ab12 key=e5b0acaa
# columns: utc_date,utc_time,lat,lon,alt_m,speed_kmh,course_deg,sats,hdop
# decrypt: tools/theoros.py decrypt-log MASTER_FILE <this file>
01020304,5UKzG0B7UMdcPyGyHNnLiLWgKZ0oFe01C3m2b/WoKowzmBvLDt010lkH3ErZ/Q2phSSxorqVBveFvoHjWFrLfBz1IE9
zSkb4hlS
```

- **Line 1:** magic # theoros encrypted log v1, then node= (4 hex digits) and key= (device key fingerprint). Decoders check the fingerprint before decrypting, to catch the wrong master file.
- **Lines starting with #** are comments.
- **Record lines:** <counter, 8 lowercase hex digits>, <base64>, where the base64 (standard alphabet, padded) encodes ciphertext || 8-byte tag. Nonce = counter || node || L; no AAD; plaintext = the CSV record without line ending (at most 160 bytes).
- One record per line, synced to the card after each, so a power cut damages at most the last line.

Decoder behaviour (theoros.py decrypt-log): lines that fail to parse or authenticate are reported as *damaged or altered* and skipped. Authentic lines whose counter does not increase are reported as *duplicated or reordered* and skipped. Lines removed entirely cannot be detected, because counter gaps are normal (the counter is shared with packets).

Without a key the tracker writes plain CSV (the columns line, then records), and the console and OLED say so.

10. Key storage on the device

Item	Storage	Notes
Key (tracker: device key; receiver: master)	NVS theoros-sec / key, 16 bytes	Plain in flash until section 16
Message counter	NVS theoros-sec / ctr	Next unreserved value
Receiver peer counters	NVS theoros-sec / p<node>	Highest accepted counter

- Keys are entered only over the USB console, never over WiFi or LoRa.
- Boards never print a key, only its fingerprint.
- Derived keys and plaintext buffers are wiped after use where practical.
- **Power-on self-test:** both firmwares run the RFC 3610 packet vector #1 (encrypt, decrypt, forged-tag rejection) and RFC 5869 test case 1. If any fails, encryption is disabled: the tracker does not transmit, and the console reports CRYPTO SELF-TEST FAILED.

Console commands:

Command	Effect
key set <32 hex digits>	Store the key, reboot
key clear	Remove the key, reboot. The counter is kept
status	Shows self-test result, key type and fingerprint

11. Provisioning

Requires Python 3.9+ and `pip install cryptography`.

1. Master key, once:

```
firmware/tools/theoros.py new-master ~/theoros.master
```

Creates the file with mode 600 and refuses to overwrite an existing one. Keep it offline and backed up (section 12).

2. Each tracker: read the node ID from its console (`status` , `lora`: line), then:

```
firmware/tools/theoros.py provision ~/theoros.master ab12
# tracker ab12, device key fingerprint 982dc4e9
key set 3f1c...
```

Paste the `key set` line into the tracker's console. After the reboot, `status` must show the same fingerprint.

3. The receiver:

```
firmware/tools/theoros.py provision ~/theoros.master receiver
```

Paste into the receiver's console and check the fingerprint the same way.

4. Clean up. Clear the terminal scrollbar; the key was typed there.

12. Operating procedures

Situation	Action
Back up the master	Two offline copies (e.g. encrypted USB stick, paper in a safe place). Without it no log or packet can ever be decrypted
Master lost	Old logs are unreadable. Create a new master and re-provision every board
Master leaked, or receiver lost	Assume all data, past and future, is readable by the finder. New master, re-provision all boards
Tracker lost	Its data is exposed; others are not. Ideally re-key everything with a new master. Per-tracker revocation is not implemented: until re-keyed, the receiver still accepts its packets
Add a tracker	<code>provision</code> with the master; no change on the receiver
Replace the receiver	Flash it, <code>provision ... receiver</code> . Its peer table starts empty, which briefly reopens the replay window (section 15)
Read a tracker's card	<code>theoros.py decrypt-log ~/theoros.master LOG0001.CSV > track.csv</code>
Decode captured packets	<code>theoros.py decode ~/theoros.master <hex> ...</code>
Retire a board	<code>key clear</code> , then erase flash (<code>esptool.py erase_flash</code>)

13. Test vectors

Standard vectors (run at boot and in host tests)

RFC 3610, packet vector #1. Key `C0C1...CF`, nonce `00000003020100A0A1A2A3A4A5`, AAD `0001020304050607`, plaintext `08090A...1E` (23 bytes):

```
ciphertext 588C979A61C663D2F066D0C2C0F989806D5F6B61DAC384
tag        17E8D12CFDF926E0
```

RFC 5869, test case 1. IKM `0B × 22`, salt `000102...0C`, info `F0F1...F9`, `L = 42`:

OKM 3CB25F25FAACD57A90434F64D0362F2A2D2D0A90CF1A5A4C5DB02D56ECC4C5BF34007208D5B887185865

Theoros vector

Inputs:

Input	Value
Master	00112233445566778899aabbccddeeff
Node	0xab12
Counter	0x01020304
Position	time 1791364364 (2026-10-07T09:12:44Z), lat 129715987, lon -775945627, alt -12, speed 1234, sats 14, HDOP×10 8, flags 0x05
Log record	2026-10-07,09:12:44,12.9715987,-77.5945627,-12.0,123.40,87.5,14,0.8

Outputs:

Output	Value
Device key	845f815fc13d4836a5e69fe7ba61c0d7
Radio key	85011cb408223713e02ed992feb10d45
Log key	3f736e8ecf96efd82b93413fb4f22425
Device key fingerprint	e5b0acaa
Radio nonce	04030201 12ab 52 000000000000
Packet	0212ab04030201676c1254840ea0e459b56b903af8cec01ae39dc3175e30bdd17100
Log line	01020304,5UKzG0B7UMdcPyYgYHNnLiLWgKZ0oFe01C3m2b/WoKowzmBvLDt010lkH3ErZ/Q2phSSxorqVBveFvoHjWFrIfBzLIE9zSkb4hLS

14. Verification

make -C firmware/tests (needs libssl-dev and Python cryptography):

- Standard vectors through the production code paths.
- Packet seal and open round trip; **each of the 34 bytes flipped in turn must be rejected**, with no plaintext released.
- Wrong key, other tracker's key, truncated packet: rejected.
- Same position with the next counter gives an unrelated ciphertext.
- Log lines: round trip; wrong node, wrong key, altered, cut off and garbage lines rejected.
- **Independent cross-check:** the firmware's keys, packet and log line are re-derived and re-encrypted with Python cryptography and must match byte for byte.

On hardware the boot self-test covers the mbedTLS backend, which cannot run on the host.

15. Limitations

- **Traffic analysis.** Transmission, timing, signal strength, node ID and counter are visible. Direction finding can locate a tracker.
- **Keys in plain flash.** A flash dump of a board yields its key. Fixed by section 16.

- **Receiver is trusted.** Its log, USB output and display hold decrypted positions, and its flash holds the master key.
- **Replay after receiver reboot.** Up to 10 minutes of recorded packets newer than the last saved counter can be replayed once each, until a genuine newer packet arrives; the freshness check limits this further once the receiver's clock is set.
- **No per-tracker revocation.** A lost tracker's key stays valid until everything is re-keyed.
- **Log line deletion** is undetectable.
- **Web page** is plain HTTP (WPA2 only). Tracker logs downloaded from it are still encrypted; the status page shows the current position.
- **Unkeyed tracker** logs in plaintext (but never transmits).
- **Not yet run on hardware.** The mbedTLS backend is only exercised by the boot self-test, which first runs at bring-up.

16. Hardening roadmap

Not done, because each step burns one-time eFuses and can brick a board if done wrong. For finished boards, in this order, after testing on one sacrificial board:

1. **Flash encryption** (release mode): the flash contents, including NVS keys, are unreadable off-chip.
2. **NVS encryption:** required alongside flash encryption for NVS partitions.
3. **Secure boot v2:** only signed firmware runs, so nobody can flash code that reads the key out. OTA images must then be signed.
4. Optionally keep the master key in an eFuse key block with the HMAC peripheral, so even the firmware cannot read it.

Follow Espressif's ESP32-S3 *Flash Encryption* and *Secure Boot v2* guides. Later possible improvements: per-tracker revocation list on the receiver, HTTPS for the service page, persisting peer counters more often with wear levelling.